SmartField v0.1

Informe de auditoría de seguridad

Auditor: División de Seguridad de Kovigroup

Fecha: 21 de marzo de 2026

Alcance: Componente de cifrado del lado del cliente (`smartfield.js` , 640 líneas)

Método: Revisión de código línea por línea + 20 vectores de ataque automatizados

Clasificación: Pública

RESULTADO: 0 Crítico · 0 Alto · 3 Medio · 4 Bajo

1. Resumen ejecutivo

SmartField es un componente web `<smart-field>` que reemplaza `<input>` los elementos HTML estándar con una alternativa cifrada. Cada pulsación de tecla se cifra mediante AES-256-GCM para el cifrado de datos y RSA-OAEP-2048 para el intercambio de claves, implementado a través de la API Web Crypto nativa del navegador.

El componente utiliza un Shadow DOM cerrado para aislar los elementos internos, un WeakMap para almacenar datos confidenciales fuera del objeto del elemento y un bloqueo de eventos integral para evitar la filtración de datos.

20/20	640	0
Ataques bloqueados	Líneas auditadas	Cuestiones críticas

Evaluación general: SmartField v0.1 demuestra una sólida arquitectura de seguridad para un prototipo. No se detectaron vulnerabilidades críticas ni de alta gravedad. El componente resiste con éxito los 20 vectores de ataque probados.

2. Resumen de los hallazgos

Gravedad	Contar	Descripción
Crítico	0	—
Alto	0	—
Medio	3	Exposición global a criptomonedas, sin aplicación de HTTPS, descifrado del lado del cliente.
Bajo	4	Referencia indefinida, método faltante, ventana de tiempo, registros de consola

3. Análisis de cifrado

3.1 Selección de algoritmos

Componente	Algoritmo	Estándar	Evaluación

Cifrado de datos	AES-256-GCM	NIST SP 800-38D	✅ **Fuerte**
Intercambio de claves	RSA-OAEP-2048	PKCS#1 v2.1	✅ **Fuerte**
Función hash	SHA-256	FIPS 180-4	✅ **Fuerte**
Generación aleatoria	cripto.obtenerValoresAleatorios()	Criptografía web del W3C	✅ **Fuerte**

3.2 Generación de claves (líneas 12-17)

```
cripto.subtle.generateKey(
  { nombre: 'RSA-OAEP', longitud del módulo: 2048,
    publicExponent: new Uint8Array([1, 0, 1]), hash: 'SHA-256' },
  verdadero, ['encriptar', 'descifrar']
);
```

✅ **Verificado:** Utiliza la API Web Crypto (acelerada por hardware, compatible con FIPS en la mayoría de las plataformas). RSA-2048 es el estándar mínimo recomendado por NIST hasta 2030. El exponente público 65537 previene ataques de exponente pequeño.

3.3 Cifrado híbrido (líneas 27-36)

```
// Cifrado por pulsación de tecla:
1. Generar una clave AES-256 aleatoria (una nueva por cada cifrado).
2. Generar un vector de inicialización (IV) aleatorio de 12 bytes (uno nuevo por cada cifrado).
3. Cifrar los datos con AES-256-GCM.
4. Cifrar la clave AES con RSA-OAEP.
5. Empaquetar como: base64(JSON{versión, iv, clave cifrada, datos cifrados})
```

✅ **Verificado:** Cada cifrado utiliza una clave AES y un vector de inicialización (IV) únicos, lo que proporciona confidencialidad directa a nivel de pulsación de tecla. AES-GCM ofrece confidencialidad e integridad (cifrado autenticado).

4. Análisis de aislamiento de datos

4.1 Shadow DOM (Línea 52)

```
this._shadow = this.attachShadow({ mode: 'closed' });
```

✅ **Verificado:** `mode: 'closed'` devuelve `null` para `.shadowRoot`. El JavaScript externo no puede acceder a los elementos internos a través de ningún método de consulta DOM.

4.2 Almacenamiento de WeakMap (líneas 46-61)

```
const _secretos = nuevo WeakMap();
_secrets.set(this, { realValue: '', encrypted: '', keys: null, ... });
```

✅ **Verificado:** Datos confidenciales (valor en texto plano, claves de cifrado, ID de campo) almacenados en un WeakMap definido en el cierre IIFE. No accesible a través de `JSON.stringify()`, `Object.getOwnPropertyNames()`, `for...in`, ni ninguna técnica de enumeración de propiedades.

4.3 Propiedad de valor no configurable (líneas 65-72)

```
Objeto.defineProperty(this, 'valor', {
  obtener: función() { devolver _secretos.get(me).encriptados; },
  conjunto: función() {},
  enumerable: falso,
  configurable: false // No se puede sobrescribir
});
```

✅ **Verificado:** `configurable: false` previene ataques de contaminación de prototipos. Un atacante no puede redefinir `.value` usando `Object.defineProperty()`.

5. Pruebas de resistencia a ataques

Se probaron 20 vectores de ataque automatizados contra el componente. Los 20 fueron bloqueados.

#	Vector de ataque	Uso en el mundo real	Resultado
1	`.value` leer	Robo de contraseñas a partir de entradas normales	**OBSTRUIDO**
2	`.shadowRoot` acceso	Acceso a los componentes internos del componente web.	**OBSTRUIDO**
3	`.innerHTML` leer	Extracción de contenido DOM	**OBSTRUIDO**
4	`.textContent` leer	Extracción de texto	**OBSTRUIDO**
5	`.innerText` leer	Extracción de texto visible	**OBSTRUIDO**
6	`querySelector("input")`	Relleno de formularios de bot	**OBSTRUIDO**
7	`querySelectorAll("*")`	enumeración de elementos	**OBSTRUIDO**
8	`children/childNodes`	Recorrido del árbol DOM	**OBSTRUIDO**
9	`outerHTML` fuga de datos	Extracción de atributos HTML	**OBSTRUIDO**

10	Metadatos (tipo/nombre/longitud)	Identificación de campo	**OBSTRUIDO**
11	`.value` inyección	Inyección de datos maliciosos	**OBSTRUIDO**
12	oyente de eventos Keydown	Captura de keylogger	**OBSTRUIDO**
13	Contaminación prototipo	Ataque de redefinición de propiedad	**OBSTRUIDO**
14	Observador de mutaciones	Observación DOM de Hotjar/FullStory	**OBSTRUIDO**
15	`getComputedStyle`	Extracción de datos basada en CSS	**OBSTRUIDO**
16	Superposición de entrada transparente	Interceptor de pulsaciones de teclas invisible	**OBSTRUIDO**
17	`execCommand` Copiar	Robo de portapapeles	**OBSTRUIDO**
18	`JSON.stringify`	Exfiltración de serialización de objetos	**OBSTRUIDO**
19	enumeración de propiedades	escaneo de propiedades de objetos	**OBSTRUIDO**
20	API de accesibilidad ARIA	Extracción de atributos de accesibilidad	**OBSTRUIDO**

6. Hallazgos de gravedad media

M1: Exposición global de objetos criptográficos (línea 639)

Problema: El objeto interno `Crypto` quedó expuesto `window.SmartField.Crypto`, incluyendo la `decrypt()` función.
Impacto: Si un atacante obtuviera una clave privada, podría usar la función de descifrado expuesta.
Estado: **CORREGIDO** — Eliminado de la exportación global.

M2: No se aplica HTTPS en la obtención de claves (línea 330)

Problema: El `encrypt-key` atributo acepta cualquier URL, incluyendo HTTP.
Impacto: Un atacante intermediario podría inyectar la clave pública de un atacante a través de HTTP.
Recomendación: Validar que la URL de la clave comience con `https://` en producción.

M3: Función de descifrado del lado del cliente (líneas 37-43)

Problema: Existe una `decrypt()` función en el código del cliente.
Impacto: No es directamente explotable (requiere clave privada), pero no debería estar en el código del cliente.

Recomendación: Eliminar de la compilación de producción. El descifrado solo debe realizarse en el servidor.

7. Hallazgos de baja gravedad

L1: Referencia indefinida en connectedCallback (línea 622)

Problema: `self._fieldId` se utilizó en lugar de `self._s('fieldId')` .
Impacto: el envío del formulario podría fallar.
Estado: **CORREGIDO**

L2: Falta el método _stopAnim (línea 635)

Problema: `disconnectedCallback` Se llamó a `this._stopAnim()` un elemento que no estaba definido.
Impacto: Error silencioso al eliminar el elemento. Sin impacto en la seguridad.
Estado: **CORREGIDO**

L3: Retardo de lectura de atributos de 100 ms (línea 320)

Problema: `sf-type` los `encrypt-key` atributos se leen después de un setTimeout de 100 ms.
Impacto: breve ventana donde el campo acepta cualquier entrada sin validación de tipo.
Recomendación: documentar como comportamiento conocido. Considere usar `attributeChangedCallback` .

L4: Instrucciones Console.log en producción (líneas 213, 324, etc.)

Problema: Las instrucciones de depuración `console.log` permanecen en el código.
Impacto: Se produce una fuga de información (tipo de campo, estado de la clave) en la consola del navegador.
Recomendación: Eliminar para la compilación de producción.

8. Evaluación de la arquitectura

8.1 Arquitectura de datos cero

SmartField opera bajo un modelo de conocimiento cero. La empresa (SmartField) nunca ve, almacena ni transmite:

- Contraseñas de usuario o datos de formulario
- Claves de cifrado (generadas en el servidor del cliente)
- Información de identificación personal (IIP)

✅ **Verificado:** El componente del lado del cliente solo se comunica con el servidor del cliente para obtener la clave pública. No se envía ningún dato a la infraestructura de SmartField.

8.2 Capas de protección

#	Capa	Mecanismo	Estado
1	Aislamiento del DOM	Sombra cerrada DOM	✅ **Activo**
2	Cifrado de datos	AES-256-GCM	✅ **Activo**
3	Intercambio de claves	RSA-OAEP-2048	✅ **Activo**
4	Aislamiento de memoria	Mapa débil (no enumerable)	✅ **Activo**
5	Bloqueo de eventos	detener la propagación (fase de captura)	✅ **Activo**
6	Protección del portapapeles	preventDefault al copiar/pegar/seleccionar	✅ **Activo**
7	Protección visual	Personajes cifrados animados	✅ **Activo**
8	Mitigación de capturas de pantalla	Cambio de visibilidad acelerado	✅ **Activo**
9	Resistencia a los bots	Shadow DOM bloquea las consultas	✅ **Activo**
10	Prevención de guardado automático	La entrada muestra el texto cifrado, no el texto plano.	✅ **Activo**
11	Ocultación de metadatos	tipo/nombre/longitud devuelven valores falsos	✅ **Activo**
12	Protección de prototipos	configurable: false en .value	✅ **Activo**
13	Protección de serialización	WeakMap es invisible para JSON.stringify	✅ **Activo**

9. Recomendaciones para la versión 0.2

1. Agregar la aplicación de HTTPS para `encrypt-key` la URL de obtención
2. Eliminar `decrypt()` la función del código del lado del cliente.
3. Eliminar todas `console.log` las declaraciones para la compilación de producción
4. Implementar la integridad de subrecursos (SRI) para la carga de scripts.
5. Añadir mecanismo de rotación de llave
6. Agregar compatibilidad con nonce de la Política de Seguridad de Contenido (CSP)
7. Minimizar y ofuscar la compilación de producción

8. Prueba de penetración realizada por terceros a cargo de Cure53 o NCC Group para la certificación empresarial.

10. Conclusión

SmartField v0.1 demuestra una sólida arquitectura de seguridad con 13 capas de protección independientes y cero vulnerabilidades críticas. La combinación de cifrado acelerado por hardware (Web Crypto API), aislamiento Shadow DOM cerrado, protección de memoria WeakMap y bloqueo integral de eventos crea un enfoque de defensa en profundidad que resiste los 20 vectores de ataque probados.

Los tres hallazgos de gravedad media (uno ya corregido) se relacionan con el endurecimiento de la producción, más que con fallos de seguridad fundamentales. Los mecanismos principales de cifrado y aislamiento están implementados correctamente según los estándares de la industria (NIST, W3C).

Este componente es adecuado para implementaciones controladas y pruebas beta públicas. Para entornos de producción empresariales que manejan datos financieros o sanitarios, se recomienda una prueba de penetración complementaria dirigida por un experto.

División de Seguridad de Kovigroup — Auditoría de Seguridad Independiente
SmartField v0.1 · Componente: `smartfield.js` (640 líneas) · Fecha: 21 de marzo de 2026 · Clasificación: Pública · Distribución: Sin restricciones